Some model tests

Author Name

Affiliation

(Dated: March 11, 2026)

Abstract

Some numerical test about training and testing of cross section fitting using neural networks

I. INTRODUCTION

We use a neural network as universal interpolator to fit calculations of total cross sections (TCS) within two theoretical models, namely the Continuum-Distorted-Wave with Eikonal-Initial -State (CDW-EIS) and the Classical Trajectory MonteCarlo (CTMC) for ionization of hydrogen by bare ion impact. Here we consider bare projectiles with charges Z_P ranging between 1 and 9

II. METHODS

A. The training set

The training set consist in a number of 1013 TCS. We have calculated 513 TCS with the CDW-EIS model and the CTMC model as it is specified in Table I. It is important to note that we have considered here the semiempiric model of Rudd [poner cita] on an equal footing with the theoretical models.

Theory	Number of Impact Energies	Impact Energy Range (keV/amu)	Ion charges
CDW-EIS	37	10 - 100000	1 - 9
CTMC	20	15 - 2500	1 - 9
Semi Empiric	500	10 - 5000	1

TABLE I. Number and energy range of TCS calculated for each model.

The input variables are, the impact energy, the projectile charge, and the theory used to obtain the TCS. Since MLPs are not suitable to handle very tiny numbers nor several orders of magnitude, as in the case of this training set, we must perform a preprocessing stage that include the data compression by taking the decimal logarithm of the impact energy and the TCS. These preprocessing allow us to avoid gradient vanishing in the backpropagation algorithm used to fit the data because using tiny floating point values for the energy and the TCS. Moreover, logarithm produce a compactacion of the several magnitude orders

that TCS present for the wide impact energies considered here. On the other hand, the categorical variable corresponding to the theory must be casted to a numerical value. We choose to expand a three dimensional One Hot Encoding for the three theories considered here.

B. The model

We show in Figure 1 a scheme of the model used in this work. After the preprocessing stage we have a MLP with a 5-dimensional input, then the hidden layers (N_L) and a single output corresponding to the $\log_{10}$ of the TCS.

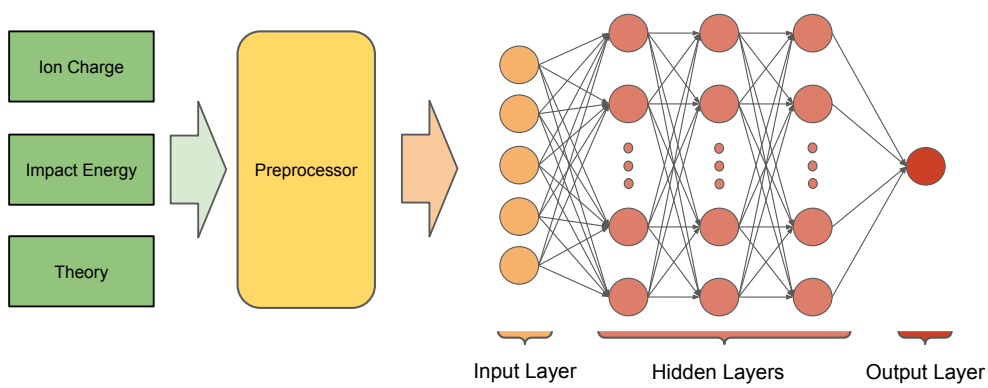

FIG. 1. Interpolator scheme

To find the best architecture of the MLP, as the training set is not big enough, we have performed averaging over a k-fold split over the training set, and the corresponding training for models with hidden layers ranging from one to five ($N_L = 1, ...5$), with different number of units per layer ($N_H = 2^n$ for $n = \{4, ..., 8\}$). As it is well known, to introduce non linearity in the MLPs, non linear activation functions must be used for each unit output. Unlike the usual trend to use the fast-to-compute Rectifier Linear Unit activation function (ReLU), in this work we use two kind of infinite differentiable functions: the hyperbolic tangent, and a gaussian activation. The choice of smooth activation functions can be easily justified by the fact that TCS are smooth functions of the impact energy, and extreme non differentiable oscillating fits caused by ReLU and its variations are naturally avoided.

This fitting process is a typical supervised learning. In supervised learning some *distance*

function between the model prediction y_{pred} and the true value y_{true} must be computed. This *distance* function, is called the loss function $\mathcal{L}_\theta$, where θ refer to the MLP parameters. In this work we explore two loss functions

$$\mathcal{L}_\theta^{LCH}(y_{true}, y_{pred}) = \log[\cosh(y_{pred} - y_{true})] \tag{1}$$

$$\mathcal{L}_\theta^{ERR}(y_{true}, y_{pred}) = L_1(y_{pred}, y_{true}) + L_2(y_{pred}, y_{true}) + Rel(y_{pred}, y_{true}) \tag{2}$$

In Eq. (2) three different metrics are added,

$$L_1(y_{true}, y_{pred}) = \frac{1}{N}\sum_{i=1}^{N} |y_{pred} - y_{true}| \tag{3}$$

$$L_2(y_{true}, y_{pred}) = \frac{1}{N}\sum_{i=1}^{N} (y_{pred} - y_{true})^2 \tag{4}$$

$$Rel(y_{true}, y_{pred}) = \frac{1}{N}\sum_{i=1}^{N} \left|\frac{y_{pred} - y_{true}}{y_{true}}\right|. \tag{5}$$

Usually, L_1, L_2, and Rel are named as the mean absolute error (MAE), the mean squared error (MSE) and the relative error, respectively. We combine these three measures in order to guide the optimization algorithm for the different ranges in the $\log_{10}(TCS)$. While on one hand MSE ponderates big differences, especially where target values are higher, the MAE does it at intermediate regions. On the other hand, Rel runs over whole range on equal footing.

As we are dealing with analytic activation functions, we can use the gradient descent algorithm or any variant of it. In this work we use the Adam algorithm with an initial learning rate of $lr = 1 \times 10^{-4}$ to *backpropagate* the model and update the parameters.

First, we want to point out that the increase of N_L above two hidden layers does not produce significant improvements. A similar behavior occur with the increasing of N_H above $N_H = 128$ as can be observed in Fig. 2. In this Figure we show the evolution of both loss function during the hyperparamenter exploration process as function of the iteration number or *epoch* of the optimization process. Moreover, we observe that $\mathcal{L}_\theta^{ERR}$ produces more soft optimization curves, and, except for the case of $N_H = 32$ with

We show in Fig. (2)

We use PyTorch framework to optimize the fitting of a multilayer perceptron (MLP) over a set of pre-calculated ionization TCS.

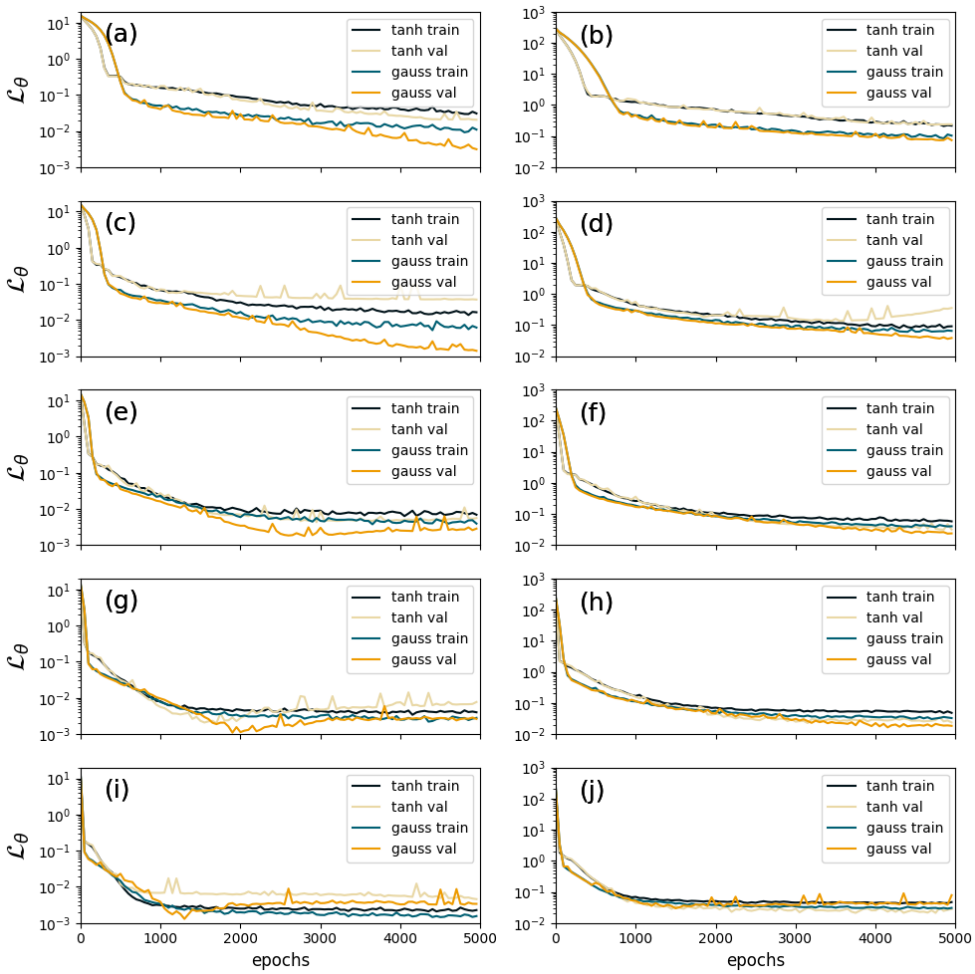

FIG. 2. Averaged in 5-folded loss functions calculated over the training and the validation sets as function of the epoch. In all the cases the number of layesrs $N_L = 2$. Left column: $\mathcal{L}_\theta^{LCH}$ loss function, right column: $\mathcal{L}_\theta^{ERR}$ loss function. Each row correspond to architectures with increasing number of hidden units $N_H = \{16, 32, 64, 128, 256, 512\}$, shown in panels (a) and (b); (c) and (d); (e) and (f); (g) and (h); and (i) and (j), respectively.

The architecture of the MLP has an input of the projectile impact energy, the charge of the projectile and the theory used

- Impact energy: we use decimal logarithm to flat the data in usable ranges ≈ 10
- Projectile charge: casted to float
- Theory: we one-hot encode the theory used (CTMC, CDW-EIS and semi-empirical formula for proton impact by Rudd et al.)

- As the number hidden units increase, the lower loss reached.
- The gaussian activation function has better performance

VA CON

We noted that increasing the number of layers become meaningless after a total of three layers. On the other hand the use of different number of units in the different layers did not improve the fitting, for these reasons we choose to employ a MLP with three hidden layers and present the study of the training performance for the case with 32, 64 and 128 units per layer.

C. Training process

For the training process we use the Adam algorithm, which is based in gradient descent algorithm. To compute distances we use

III. RESULTS

A. Training process

In Fig. 4 we show the evolution during the training process for several global metrics that compute the error

- As the number hidden units increase, MAE and MRE seems to converge for gaussian activation.
- tanh present some overfitting.
- The gaussian activation function has slightly better performance

In Fig. **??** we show the evolution during the training process for several maximum of the former metrics to understand the deviations over each model

- This plot shows the power of gaussian activation.

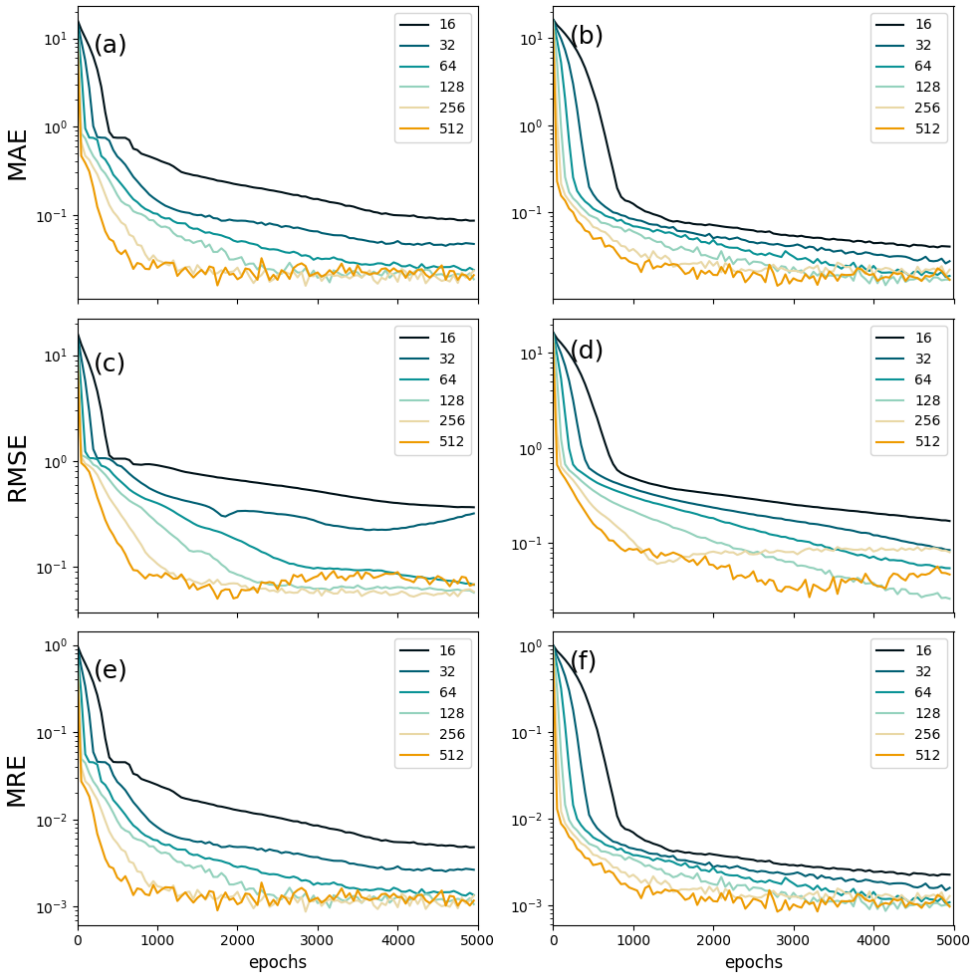

FIG. 3. Metrics over validation set for different network architectures. Left column (a, c, and e) metrics with tanh activation function. Right column (b-d-f) gaussian activation function. Each row correspond to a different metric over the $\log_{10}$ of the TCS.

B. Evaluation stage

We perform some evaluations over the whole TCS training set with the best models (128 hidden units) to understand the different errors in cm^2. Even when we have determined that gaussian activation has better performance, we show results for both activation types

- This plot shows the power of gaussian activation.

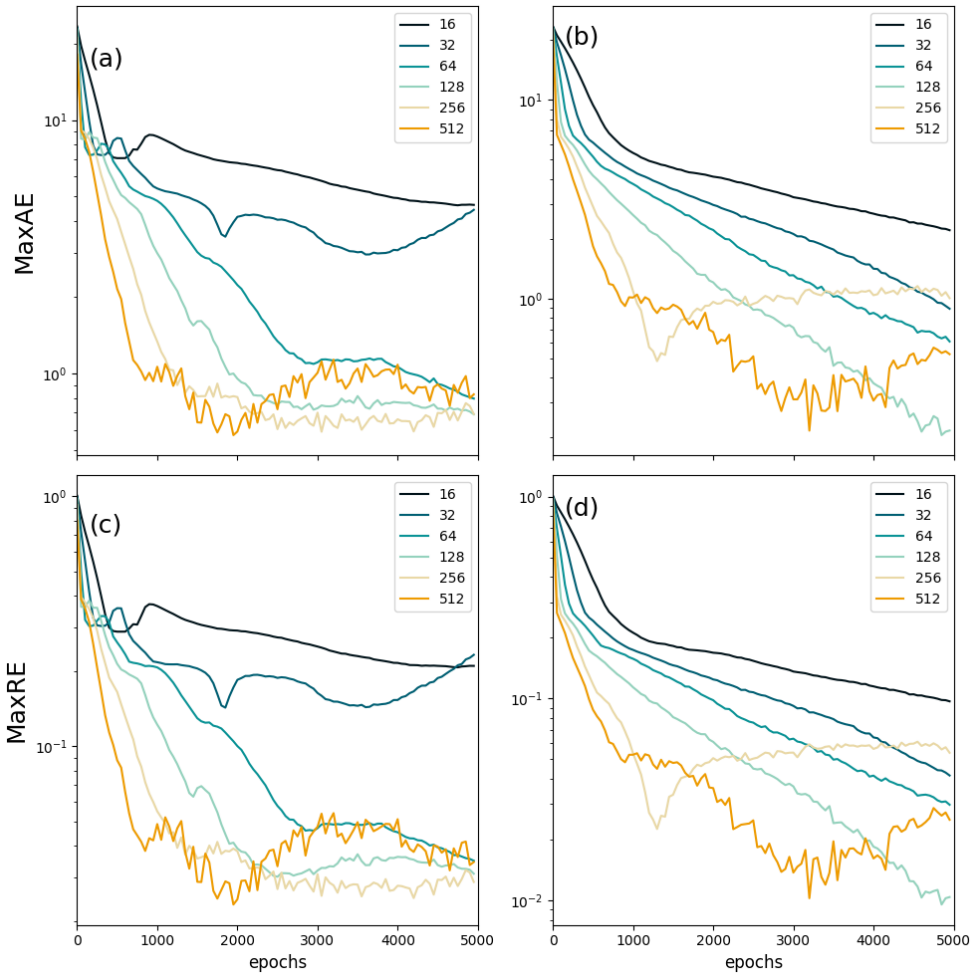

FIG. 4. Maximum values of Metrics over validation set for different network architectures. Left column (a, c) metrics with tanh activation function. Right column (b-d) gaussian activation function. Each row correspond to a different metric over the $\log_{10}$ of the TCS.

IV. DISCUSSION

- WE NEED TO DO THE EXPERIMENT WITH ZP HOLES AND ENERGY EXTRAPOLATIONS
- WE NEED TO DEFINE RANGES OF INPUT ENERGIES IN THE INPUT PIPELINE
- SEBASTIAN HAS TO PLOT THE SEMIEMPIRIC FIT!!!!
- MISSING THE BENCHMARKING AGAINST THE CUBIC INTERPOLATOR. ASK TO MICHELE IN DEEP.

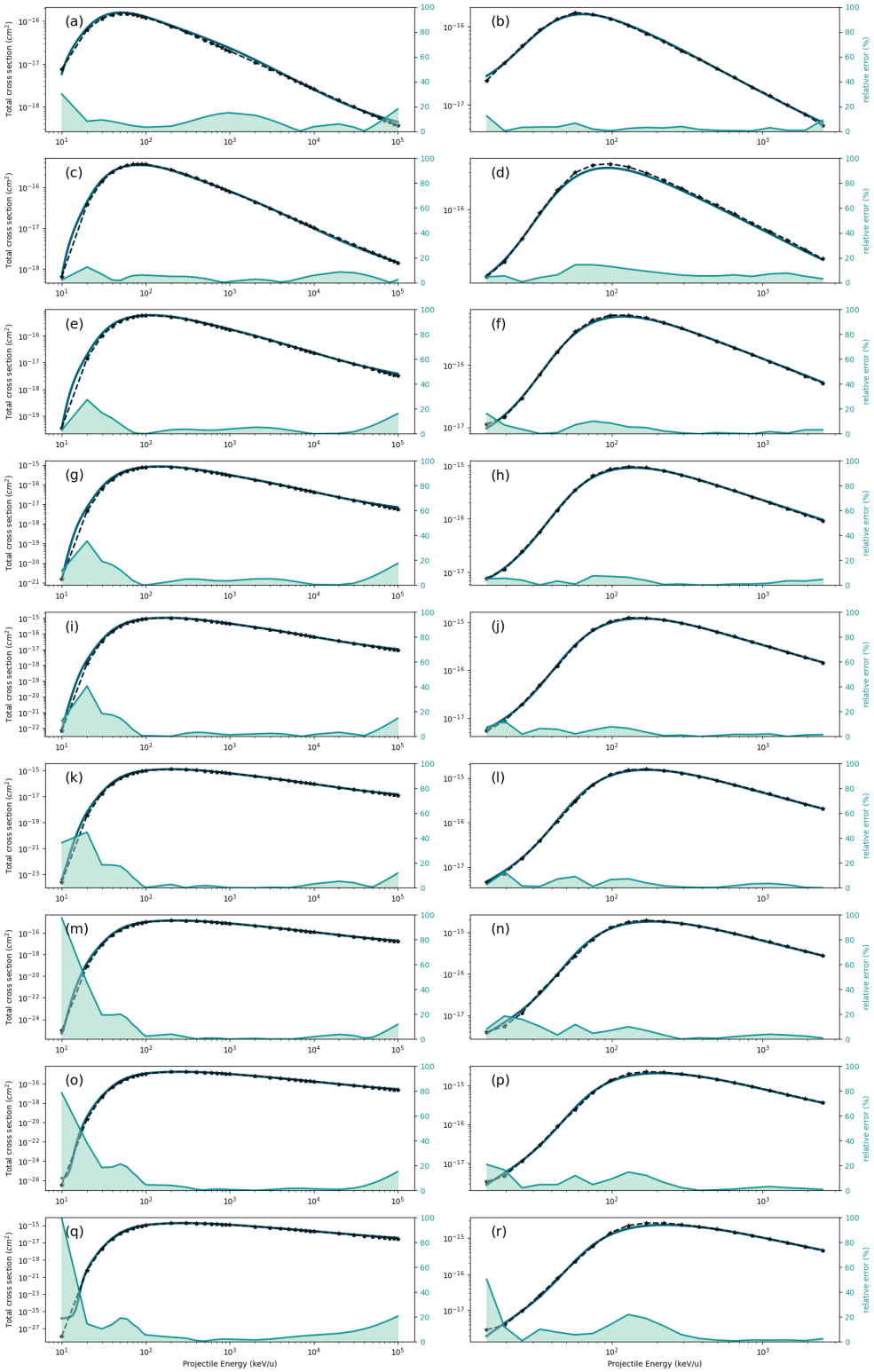

FIG. 5. Prediction of the MLP for CDW-EIS (left column) and CTMC (right column) as a function of the projectile impact energy. Each row correspont to a different projectile charge from $Z_P = 1$ to $Z_P = 9$. Activation function: tanh

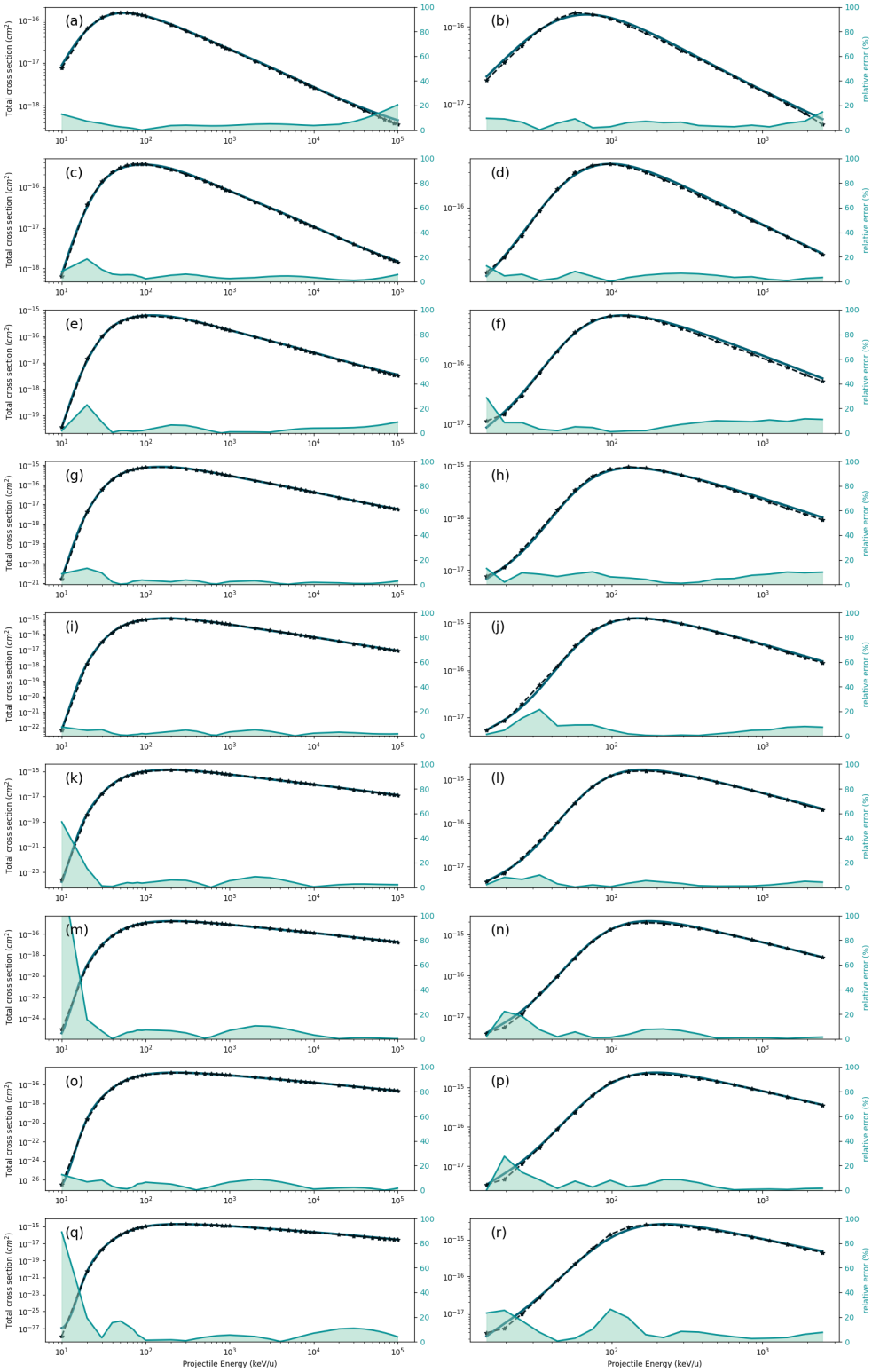

FIG. 6. Prediction of the MLP for CDW-EIS (left column) and CTMC (right column) as a function of the projectile impact energy. Each row correspont to a different projectile charge from $Z_P = 1$ to $Z_P = 9$. Activation function: gaussian.

V. CONCLUSION

Summarize your conclusions and future work.

[1] Author, "Title," *Journal*, vol. X, pp. XX-XX, Year.